

Cours 17 — Le tri par insertion · Terminaison

Semaine 17 ■ Manuel Hachette ch. 6 (§ 6.3) ■ 2×1 h

Objectifs

- Dérouter et programmer le tri par insertion
- Comparer insertion et sélection : meilleur cas, pire cas
- Justifier qu'une boucle `while` se termine (variant)

1. L'idée : insérer dans du déjà trié

C'est le tri du joueur de cartes : chaque nouvelle carte est **insérée à sa place** dans l'éventail déjà trié.

Sur un tableau : à l'étape i , la zone $t[0:i]$ est triée ; on prend $t[i]$ et on le fait **glisser vers la gauche** jusqu'à sa place, en décalant les plus grands d'un cran à droite.

$[3, 5, 1, 4] \rightarrow [3, 5, 1, 4] \rightarrow [1, 3, 5, 4] \rightarrow [1, 3, 4, 5]$ — trié.

2. Le programme

```
def tri_insertion(t):
    """Trie le tableau t en place (ordre croissant).

    MODIFIE t. Ne renvoie rien.
    """
    for i in range(1, len(t)):
        valeur = t[i]                # la carte à insérer
        j = i
        while j > 0 and t[j - 1] > valeur:
            t[j] = t[j - 1]          # décaler à droite
            j = j - 1
        t[j] = valeur                # insérer
```

L'ordre des conditions du `while` compte : $j > 0$ **avant** $t[j - 1] > \text{valeur}$. Python évalue de gauche à droite et s'arrête dès que le résultat est connu (évaluation *paresseuse*) : si $j == 0$, il ne tente jamais $t[-1]$. Dans l'autre ordre : lecture de $t[j - 1]$ avec $j = 0$... qui en Python lit silencieusement la dernière case — bug sournois !

3. Insertion contre sélection

Comptons les comparaisons du `while` :

	meilleur cas	pire cas	tous les cas
Sélection	$n^2/2$	$n^2/2$	identique
Insertion	$n - 1$ (déjà trié)	$n^2/2$ (ordre inverse)	ça dépend

Si le tableau est **déjà trié**, chaque `while` s'arrête immédiatement : coût **linéaire**. L'insertion est excellente sur des données *presque triées* (cas fréquent en pratique : ajouter quelques valeurs à une liste triée). Les deux restent $O(n^2)$ au pire.

4. Cette boucle s'arrête-t-elle ? Le variant

Pour un **for**, la terminaison est évidente. Pour un **while**, il faut un argument : le **variant de boucle** — une quantité entière, positive, qui **décroît strictement** à chaque tour.

Pour le **while** de l'insertion : le variant est j . Il vaut au plus i au départ, chaque tour fait $j = j - 1$, et la boucle exige $j > 0$: au bout d'au plus i tours, elle s'arrête.

```
while n != 1:                # Syracuse (TP 2) : quel variant ?  
    ...
```

Personne n'en connaît de variant — c'est pour cela que la terminaison de Syracuse est un problème ouvert ! Le variant est l'outil qui transforme « ça a l'air de s'arrêter » en **preuve**.

À retenir

- Insertion : zone gauche triée, on fait glisser $t[i]$ à sa place par décalages.
- **while** $j > 0$ **and** ... : l'ordre des conditions protège l'accès $t[j - 1]$.
- Meilleur cas linéaire (déjà trié), pire cas $n^2/2$; sélection : toujours $n^2/2$.
- Terminaison d'un **while** : exhiber un **variant** — entier, positif, strictement décroissant.